Universidad de Jaén
Departamento de Informática

Actividades Tema 3

Alta Disponibilidad, Escalabilidad y Tolerancia a Fallos

Octubre, 2026

v 1.0

Contenidos

Introducción **2**
Descripción de las actividades propuestas **2**
Actividad 3.1.- Despliegue de una arquitectura distribuida con un balanceador de carga y configuración dinámica de nodos 3
Actividad 3.2.- Descubrimiento de servicios, ajuste de recursos y pruebas de alta disponibilidad con un balanceador de carga 6
Actividad 3.3.- Pruebas de tolerancia a fallos 10
Referencias recomendadas **13**

Introducción

En las actividades propuestas para este tema trataremos de profundizar y experimentar en alguno de los patrones de diseño y técnicas estudiadas, como pueden ser el descubrimiento de servicios, su configuración dinámica, la mejora de la respuesta de una aplicación en la nube frente a situaciones de aumento de la carga de trabajo requerida o si se producen fallos en la conectividad de la red o caídas de nodos específicos. En particular, estudiaremos una de las técnicas que mayores beneficios puede aportar en este sentido como es el balanceo de carga.

Para la aplicación de las técnicas estudiadas, y como se ha visto a lo largo del tema, es necesario utilizar arquitecturas distribuidas, para lo que necesitaremos disponer de un entorno de pruebas donde poder lanzar diferentes microservicios interconectados a través

de una red. Para ello, utilizaremos en este caso contenedores con *Docker* para lanzar cada uno de los microservicios de nuestro despliegue distribuido y *docker compose* como solución de orquestación local para el despliegue e interconexión entre los mismos.

Descripción de las actividades propuestas

Preparación del entorno de experimentación

Para la realización de los experimentos propuestos se partirá del material de trabajo disponible en el repositorio de github de la asignatura en la dirección https://github.com/jrbalsas/asnwebapps[1]. En este apartado presentamos algunos de los materiales que se utilizarán para familiarizarnos con ellos y los pasos habituales que deben seguirse durante la realización de las actividades propuestas más adelante. *IMPORTANTE: esta parte de la actividad no debe incluirse en los informes; sirve únicamente para verificar que funciona correctamente la instalación de docker y familiarizarse con los conceptos y operaciones básicas de despliegues locales de microservicios utilizando contenedores.*

El fichero *compose.yml*[2] de la carpeta *loadbalancer* en el repositorio de github define un despliegue con tres contenedores para pruebas de balanceo. Uno de ellos es un balanceador de carga **traefik** que da acceso a 2 réplicas de un mismo contenedor con una aplicación web para pruebas[3] cuya única función es mostrar información sobre el propio contenedor y cada petición web que recibe. Realizar los siguientes pasos para comprobar el funcionamiento del sistema:

1. Necesitaremos **tres terminales** para estudiar el funcionamiento del sistema. En un terminal ejecutaremos el comando '*docker stats'* o, mejor aún, la aplicación '*ctop'*[4] para monitorizar el estado de los contenedores[5].
2. En otro terminal acceder a la carpeta del proyecto y realizar el despliegue con el comando '*docker compose up -d'*[6]. Una vez lanzados, utilizar en esta terminal el comando *'docker compose logs -f'* para monitorizar los logs de funcionamiento de cada contenedor
3. El tercer terminal lo utilizaremos para interaccionar con los contenedores o lanzar pruebas. Una vez realizado el despliegue y comprobado que los contenedores están lanzados y generando logs en el segundo terminal comprobaremos el funcionamiento del sistema. Para ello, abrimos un navegador y nos conectamos al puerto publicado del balanceador en la dirección *http://localhost/*. Si todo ha ido bien, veremos los datos de salida que genera la aplicación. Podemos probar a recargar la

[1] Si se dispone ya de una copia local del repositorio es conveniente comprobar si existen actualizaciones más recientes con el comando `git pull`
[2] Por compatibilidad con versiones anteriores, también se acepta el nombre *docker-compose.yml*
[3] *traefik/whoami. Consultar opciones de uso en https://hub.docker.com/r/traefik/whoami*
[4] Disponible en https://ctop.sh
[5] También puede utilizarse cualquier herramienta gráfica como el panel de control de docker desktop o el plugin de docker para VSCode.
[6] En versiones antiguas de docker era necesario instalar y utilizar la herramienta docker-compose e.g. *$ docker-compose up -d*

página varias veces y veremos cómo van generando la respuesta los diferentes contenedores

4. Los balanceadores suelen disponer de una interfaz de administración web para consultar su estado. En el caso de *traefik* para el ejemplo suministrado puede consultarse en la dirección *http://localhost:8080*. Acceder al apartado *HTTP services* y seleccionar la entrada correspondiente al servicio *wwwserver* para comprobar el estado de los nodos balanceados.
5. Al finalizar las pruebas, parar el despliegue con el comando *'docker compose down'* para liberar los puertos del host utilizados por los contenedores.
6. Observar los detalles de configuración del fichero *compose.yml* para lanzar los diferentes contenedores y las réplicas de la aplicación web. Observar también las características del fichero de configuración de cada contenedor para comprender su estructura.

Actividad 3.1.- Despliegue de una arquitectura distribuida con un balanceador de carga y configuración dinámica de nodos

En esta actividad realizaremos el despliegue en alta disponibilidad de un prototipo de aplicación web clásica conectada a una base de datos relacional, *mariadb*. Para ello partiremos del despliegue de la aplicación disponible en la carpeta phpwebappv2[7]. En el fichero *compose.yml* podemos observar cómo se lanza un contenedor con la propia aplicación y otro con la base de datos. Se ha proporcionado información al contenedor de la base de datos mediante variables de entorno para que cree una base de datos y unas credenciales que permitirán a la aplicación acceder a la misma.

Una vez que probemos el funcionamiento de la aplicación, adaptaremos el despliegue modificando el fichero compose.yml para replicar los frontales web de la aplicación e introducir un balanceador de carga. Para ello realizaremos los siguientes pasos:

Paso 1.- Prueba de despliegue con microservicios de aplicación web y base de datos

Lanzar el despliegue de la aplicación phpwebappv2 y comprobar que la aplicación responde correctamente a través de un navegador. Puede ser necesario esperar unos instantes antes de acceder a la aplicación hasta que la BD mariadb se inicialice correctamente. Consultar los logs del despliegue.

[7] esta aplicación web es una versión de la aplicación phpwebapp en el mismo repositorio que utiliza una base de datos mysql externa en vez del motor de bases de datos sqlite utilizado en la versión inicial.

Paso 2.- Configuración dinámica de microservicios con variables de entorno

Para **evitar el uso inseguro de credenciales** de acceso a la base de datos en el código de la aplicación web, enviaremos dichas credenciales de forma dinámica a la aplicación utilizando variables de entorno. Observar cómo se pasan variables al contenedor de la base de datos y utilizar un enfoque similar para enviar a la aplicación web el nombre de usuario y contraseña para que pueda conectarse a la base de datos.

A continuación, es necesario adaptar la aplicación web para recuperar dichas credenciales de las variables de entorno. En el caso de usar php, estas variables pueden recuerarse desde el vector $*_ENV, e.g. $_ENV['variable_entorno'],* o usando la función *getenv().* Como el despliegue con docker compose permite a cada frontal acceder directamente a la carpeta donde se encuentra el código de la aplicación, cada vez que se modifique su código solo hay que recargar la url de la aplicación en el navegador para observar los cambios.

Después **eliminaremos la información de credenciales de acceso del fichero de despliegue compose.yml** ya que estas podrían quedar expuestas si se almacenara en un repositorio de código externo como github. Para ello moveremos la declaración las variables de entorno que contengan información sensible como usuarios y claves de acceso a un fichero externo[8] desde donde serán leídas al realizar el despliegue.

Los **ficheros de variables de entorno**[9] suelen tener la extensión .env y almacenan en cada línea entradas del tipo VARIABLE=VALOR. Por defecto, docker obtiene valores para variables de entorno del fichero oculto *.env* si este existe en la carpeta desde donde se lanza el despliegue, pero es preferible indicar expresamente el nombre de dicho fichero dentro del propio fichero de despliegue usando la opción *env_file: "fichero.env"* asociada a los diferentes contenedores que los necesiten. Una vez realizados estos cambios, se puede forzar que docker compose recargue la configuración con *docker compose up -d* y verificar que el valor de las variables de entorno se pasa adecuadamente a los contenedores.

Paso 3.- Configuración de un balanceador de carga

Añadir al despliegue un **nuevo microservicio con un balanceador de carga** traefik como se estudió en la fase de preparación. Para ello hay que adaptar el fichero *compose.yml* con los datos del nuevo servicio balanceador. Será necesario copiar la carpeta con la configuración inicial del balanceador a la carpeta del proyecto de la aplicación web que después adaptaremos convenientemente. También es necesario indicar que se necesitarán lanzar inicialmente 2 réplicas de los frontales web, utilizando la propiedad *scale:* de un contenedor. Es importante observar que, cuando se lanzan varias réplicas de un contenedor no podemos indicar el puerto externo donde escuchará el contenedor ya que cada réplica

[8] Para proteger estos ficheros con información sensible, si el despliegue formara parte de un repositorio de código, sería necesario añadirlos al fichero .gitignore del repositorio para evitar que se publicaran en el mismo.

[9] En la documentación de docker se pueden consultar más detalles sobre tratamiento de variables de entorno

necesitará uno diferente. Por lo tanto, en la propiedad *ports:* del fichero compose.yml deberemos indicar únicamente el puerto del contenedor y NO dónde se publica en el host, e.g. solo indicar "80" (en vez de indicar el puerto del host y el del contenedor como se hacía con un único contenedor, e.g. "8080:80"). De esta forma, el puerto 80 de cada réplica se publicará en un puerto aleatorio diferente del host que esté libre y que servirá en caso de que queramos acceder directamente a la réplica en vez de hacerlo a través del balanceador.

A continuación es necesario **modificar el fichero de configuración del balanceador** para indicarle los nombres DNS de host de cada frontal web para que pueda reenviarle las peticiones web. Para ello usaremos el nombre que docker compose asigna a cada réplica de la forma: *proyecto-servicio-N*[10], donde *proyecto* es el nombre de la carpeta desde donde se lanza el despliegue; *servicio*, el nombre del servicio desde el que se lanza el contenedor tal y como aparece en el fichero compose.yml y N, que es el número de réplica. Es decir, en nuestro caso, por ejemplo, la primera réplica será phpwebappv2-wwwserver-1 .

Una vez modificada la configuración, es necesario al menos recargar el balanceador *docker compose restart traefik,* para que los cambios surtan efecto. Comprobar que los diferentes frontales web van respondiendo alternativamente accediendo al balanceador con un navegador. Observar tanto la salida del navegador como los logs de funcionamiento de los contenedores.

Nota importante: Dependiendo del balanceador empleado y de su configuración por defecto, es posible que, al lanzar el despliegue, si el balanceador se ejecuta antes que los frontales, los marque como inactivos puesto que no respondan todavía. En ese caso, el balanceador responderá con un código de error, e.g. 503, ya que no encuentra ningún frontal. Si esto ocurriera, una solución temporal es reinicializar el servicio del balanceador, e.g mediante el comando *docker compose restart traefik* puesto que al hacerlo ahora sí detectará correctamente a los frontales. En la actividad siguiente veremos cómo solventarlo de forma más elegante a nivel de configuración del balanceador.

Para la **entrega de la actividad** se proporcionarán los siguientes materiales:

- Breve **informe** (2-3 páginas) donde se aporten evidencias: capturas de pantalla, logs, comentarios, etc. que reflejen que se ha conseguido desplegar y hacer funcionar correctamente el sistema.
- Fichero comprimido de la **carpeta del proyecto** con todos los recursos exclusivamente[11] necesarios para poder reproducir su funcionamiento: ficheros *Dockerfile* que sean necesarios, *compose.yml*, fuentes de la aplicación desplegada y ficheros de configuración adicionales necesarios para personalizar y lanzar los diferentes microservicios utilizados. Además, se incluirá un fichero README.md con instrucciones para construir, lanzar y probar el acceso a la aplicación.

[10] Si se utiliza el ejecutable antiguo docker-compose, los nombres de los contenedores se generan con guión bajo: proyecto_servicio_N. En cualquier caso comprobar los nombres reales asignados a los nodos balanceados con: `docker-compose ps` o `docker compose ps`

[11] NO enviar toda la carpeta del repositorio asnwebapps local, solo los ficheros necesarios para reproducir el experimento indicado

Se valorará positivamente la utilización otra aplicación web diferente a phpwebapp, como la aplicación de pruebas realizada en la actividad 2.4 en caso de que fuera operativa o de algunas de las alternativas en el repositorio asnwebapps, siempre que éstas estén debidamente conectadas con la base de datos mariadb .

Tiempo estimado: 1-2 horas (considerando la consulta de información y pruebas)
(**plazo de entrega: 22 de octubre**)

Actividad 3.2.- Descubrimiento de servicios, ajuste de recursos y pruebas de alta disponibilidad con un balanceador de carga

En este apartado comprobaremos el rendimiento del sistema utilizando la herramienta en línea de comandos '*ab', Apache benchmark,* disponible en Ubuntu en el paquete *apache2-utils*. Esta herramienta nos ayudará a ajustar los recursos de nuestro systema observando cómo se comporta de forma controlada frente a cargas ficticias. La herramienta *ab*[12] permite lanzar a un servidor muchas peticiones web de forma simultánea, durante un tiempo determinado, para conseguir simular condiciones de carga habituales. Además, la herramienta genera un resumen con los resultados obtenidos indicando datos como el número de peticiones válidas realizadas (estados 2xx), número de peticiones no válidas (otros estados, e.g. 5xx, 4xx, etc.) y diferentes indicadores de tiempo que permiten conocer cómo se ha comportado el servidor durante la prueba.

Paso 1.- Descubrimiento de nodos balanceados

Las aplicaciones en al nube deben ser elásticas, es decir, capaces de adaptarse automáticamente a cambios en el entorno. Por lo tanto, si fuera necesario adaptar el número de nodos balanceados para atender a la demanda de peticiones, el balanceador debería poder detectar nuevos nodos o nodos que ya no están disponibles para, respectivamente, poder enrutarles, o dejar de hacerlo, peticiones. Algunos balanceadores como Traefik permiten adaptar dinámicamente el tráfico a los nodos consultando dinámicamente el gestor de recursos utilizados, e.g. docker.

En primer lugar vamos a cambiar el proveedor de información (provider) de traefik a *docker* descomentando la entrada correspondiente en su fichero de configuración, traefik.yml. También comentaremos la configuración mediante archivo ya que no la vamos a utilizar a partir de ahora (provider.file)

```
providers:
#  file: #back-end static servers configuration
#    filename: /etc/traefik/dynamic-conf.yml
 docker: # {} # enable docker dynamic container detection
```

[12] Ejecutando *ab* sin parámetros nos mostrará las diferentes opciones de configuración de que dispone, aunque para estas actividades solo utilizaremos las opciones -c y -t

```
      # Process only containers with label "traefik.enable=true"
      exposedByDefault: false
```

A partir de este momento, Traefik consultará el estado de los nodos que debe balancear directamente desde docker[13] y les configurará el enrutamiento del balanceador usando las etiquetas que estén vinculadas a dichos contenedores, en concreto a los que tengan la etiqueta *"traefik.enable=true"* en compose.yml

Realizar los cambios propuestos, parar y relanzar el despliegue, y comprobar tanto el funcionamiento de la aplicación en el navegador, como que traefik detecta dichos nodos en el panel de administración.

Para comprobar la detección automática del número de nodos balanceados, probar a cambiar el número de réplicas con el comando *docker compose scale wwwserver=5* y verificar en el panel de administración que se detectan los cambios en el número de réplicas [14].

Paso 2.- Ajuste de recursos de los nodos balanceados

En todo sistema en la nube es fundamental que los servicios estén debidamente ajustados para atender peticiones atendiendo a los recursos de los que disponen. Esto quiere decir que, si en un momento determinado se demandan al sistema más peticiones de las que este puede atender, el sistema no intente consumir más recursos de los que tiene asignados. En ese caso, el sistema podría quedar bloqueado debido, por ejemplo, a quedarse sin memoria física o que el nivel de carga de la CPU sea muy elevado. En ambas situaciones el sistema podría dejar de responder, por lo que intentaremos evitarlo consiguiendo al menos que solo se descarten algunas de las peticiones o que incluso tardaran más en procesarse aunque al final pudieran atenderse.

En este ejemplo ajustaremos el número de procesos simultáneos que los servidores web apache de la actividad 3.1 pueden operar (es decir, el número máximo de peticiones que se pueden atender) dependiendo de la memoria disponible. En el caso del contenedor de la aplicación phpwebappv2, el servidor web está configurado por defecto en modo *prefork* (un proceso por cada petición) y permite un máximo de 150 peticiones simultáneas[15]:

```
#fichero /etc/apache2/mods-available/mpm_prefork.conf
…
MaxRequestWorkers    150
```

Sin embargo, en el despliegue definido en docker compose se han limitado tanto el uso de cpu como el de memoria máxima de dicho contenedor para poder realizar los siguientes experimentos en un equipo habitual de trabajo:

```
      cpus: .25
```

[13] A través del fichero */var/run/docker.sock* del equipo anfitrión
[14] Puede ser necesario recargar la página del panel de administración para ver los resultados
[15] Podemos consultarlo, con el sistema lanzado, ejecutando: `docker exec phpwebappv2-wwwserver-1 cat /etc/apache2/mods-available/mpm_prefork.conf` o accediendo al sistema de archivos del contenedor desde la extensión de docker de vscode

```
    mem_limit: 48m
    memswap_limit: 48m  # not swap allowed
```

En estas condiciones, lanzamos el despliegue de la actividad anterior y tratamos de simular muchas conexiones simultáneas directamente sobre uno de los frontales[16], e.g. realizar peticiones usando 40 conexiones simultaneas durante 20 segundos

```
$  ab -c 40 -t 20 http://localhost:32923/
```

Si consultamos el uso de recursos mientras realizamos el experimento, e.g. `docker stats`, veremos como en cierto momento el contenedor trata de superar el máximo de memoria asignada y docker lo parará, lo que equivaldría a una caída del sistema[17]

Para evitar esta situación, vamos a ajustar de forma empírica el número máximo de procesos que atenderá el contenedor modificando el fichero de configuración de apache citado anteriormente. Para que el cambio sea permanente, lo realizaremos en el fichero de configuración de la imagen del contenedor, *Dockerfile*, donde podemos observar una línea que permite modificarlo al crear la imagen:

```
# Adjust max apache clients (aprox. 37m of memory use )
# RUN sed -i 's/MaxRequestWorkers.*150/MaxRequestWorkers\t30/' \
                                    /etc/apache2/mods-available/mpm_prefork.conf
```

Descomentaremos la línea en cuestión, pararemos (down) el despliegue de docker compose, forzamos a que se vuelva a construir la imagen del frontal (build) del contenedor con los cambios indicados y lo volveremos a lanzar (up). Ahora, si repetimos el experimento con *ab*, podemos comprobar como, aunque el contenedor estará al límite de memoria durante la ejecución, si el ajuste es adecuado, no debería caer como en el caso anterior, ya que el número de procesos máximo nunca llega a superar los recursos asignados. También podemos ver cómo apache va creando procesos para atender las peticiones con el comando: *`watch -n 0.5 docker compose top`* o lanzando el comando *`top`* desde una terminal del propio contenedor

Paso 3.- Pruebas de estrés sobre el sistema balanceado

Una vez ajustados los recursos de los nodos, estudiaremos cómo se comporta el sistema con el balanceador. Para ello, realizar los siguientes experimentos sobre el despliegue efectuado en la actividad 3.1, trabajando con dos nodos balanceados, prestando atención al consumo de recursos (memoria y cpu) de los contenedores afectados así como a los logs que generan:

- Lanzar 10 peticiones simultáneas durante 20 segundos a la **dirección del balanceador** usando los frontales balanceados mediante el comando:

[16] Obtener el puerto aleatorio asignado al frontal, e.g. usando `docker compose ps`, para poder acceder de forma directamente sin pasar a través del balanceador

[17] El número de peticiones simultáneas dependerá de los recursos locales del equipo anfitrión por lo que este podría ser superior, e.g. 100, para forzar a que el contenedor se detuviese.

```
$  ab -c 10 -t 20 http://localhost/
```

- Lanzar 10 peticiones simultáneas durante 20 segundos **a un único frontal** a través de su puerto de acceso directo. El puerto aleatorio para acceso directo a cada frontal puede obtenerse con el comando '*docker compose ps*'

- Repetir ambos experimentos para 20 peticiones simultáneas

Recopilar y comentar brevemente los resultados obtenidos, observando especialmente el total de peticiones atendidas, el número de peticiones erróneas (si las hay), el promedio de tiempo de conexión total (*mean*), la mediana del tiempo de respuesta (*median*, que indica el tiempo para el que la mitad de conexiones están por debajo del mismo) y el tiempo máximo empleado en atenderse una petición. Comentar también las observaciones realizadas respecto al consumo de recursos de los contenedores durante la realización de cada experimento.

> ***Importante:*** *a la hora de lanzar despliegues multi-contenedor en local debe tenerse en cuenta el impacto en el experimento de los recursos utilizados por la propia herramienta ab que afectará negativamente a los recursos disponibles para los propios contenedores. Si es posible, para obtener unos resultados más objetivos, lo ideal sería lanzar el comando ab desde otra máquina física en la misma subred de la máquina donde están desplegados los contenedores*

Para la **entrega de la actividad** se proporcionarán los siguientes materiales:

- Breve **informe** (3-4 páginas) donde se comenten brevemente los resultados obtenidos y se aporten evidencias: capturas de pantalla, logs, comentarios, etc. que reflejen que se ha conseguido desplegar y hacer funcionar correctamente el sistema. Indicar el modelo de procesador utilizado en la máquina anfitrión, e.g. *Intel i9-14900F* dónde se hayan realizado los experimentos.
- Fichero comprimido de la **carpeta del proyecto** con todos los recursos necesarios para poder reproducir su funcionamiento: diferentes ficheros Dockerfile que sean necesarios, compose.yml, fuentes de la aplicación desplegada y ficheros de configuración adicionales necesarios para personalizar y lanzar los diferentes microservicios utilizados. Además, se incluirá un fichero README.md con instrucciones para construir, lanzar y probar el acceso a la aplicación. Se puede aportar el mismo despliegue de la tarea 1.

Tiempo estimado: 1 hora (considerando que se parte del despliegue operativo de la actividad 3.1)

(**plazo de entrega: 22 de octubre**)

Actividad 3.3.- Pruebas de tolerancia a fallos

En esta tarea comprobaremos si el sistema es capaz de comportarse adecuadamente frente problemas habituales que pudieran ocurrir durante su funcionamiento, como la caída eventual de uno de los nodos balanceados o tratar de prevenir la caída del sistema completo por un aumento inusual de la demanda e uso.

Paso 1.- Recuperación frente a la caída de un nodo balanceado

En determinadas circunstancias, podría ocurrir que uno de los nodos balanceados deja de funcionar, ya sea porque el nodo se bloquea de forma temporal, permanente, o bien porque se le da de baja en el sistema. En esta situación, el balanceador debería de detectar esta situación y dejar de enviar peticiones a dicho nodo de forma permanente o, al menos, hasta que volviera a estar activo.

Para ello, simularemos esta situación parando uno o varios de los contenedores y comprobando si el balanceador es capaz de detectarlos y descartarlos mientras dure esta situación. Un contenedor del despliegue puede pararse/iniciarse temporalmente utilizando el comando '*docker stop/start nombre_contenedor*', e.g *docker stop phpwebappv2-wwwserver-3*[18].

Realizar el siguiente experimento sobre el despliegue realizado en la actividad 3.2 con dos nodos balanceados:

- Comprobar que todos los frontales funcionan realizando varias recargas en el navegador del sitio web balanceado
- Parar uno de los contenedores (*stop*) y comprobar que el balanceador es capaz de detectarlo y redirigir las peticiones a los frontales activos. Esto puede comprobarse recargando la página en el navegador varias veces para ver que siguen respondiendo los frontales activos y también en la web de administración
- Simularemos ahora un nodo que deja de responder y para ello lo pasaremos con el comando *docker pause nombre_contenedor_i.* En este estado el contenedor sigue disponible pero deja de responder a las peticiones HTTP (L7). Observaremos que por defecto traefik no es capaz de detectar su estado y las peticiones que enrute al mismo quedarán en espera hasta que se vuelva a activar
- Transcurridos unos instantes, volver a activar el frontal detenido (*docker unpause*) y comprobar, recargando la página, si el balanceador es capaz de detectar esta situación y volver a enviar peticiones al frontal recuperado.

Para que traefik detecte por su cuenta si un nodo balanceado está operativo antes de enviarle peticiones, es necesario **ajustar los parámetros de comprobación de**

[18] Para parar el contenedor, también puede hacerse usando el panel de docker desktop o el plugin de docker de VSCode o accediendo al menú contextual del contenedor en la herramienta ctop, seleccionandolo con el cursor, pulsando enter y eligiendo la opción pause/unpause respectivamente

disponibilidad (*health check*)[19]. Estudiar en la documentación de Traefik cómo realizar los ajustes de Health Check para los frontales e incluirlas como etiquetas junto al contenedor como se estudió en la actividad 3.2. En lo referente a la configuración del estado de cada nodo, centrarse en los elementos para indicar la url de comprobación, e.g. / en nuestro caso, el tiempo a esperar antes de considerar que el nodo no responde y entre comprobaciones. Utilizar valores bajos, e.g. 5s, para que sea cómodo verificar el resultado.

Cada vez que se modifiquen las etiquetas de configuración en el fichero compose.yml es necesario relanzar dicho contenedor para que la configuración surta efecto, mediante el comando *up* de docker compose, e.g. *'docker compose up -d wwwserver'.* Los logs de los contenedores darán información sobre si hay algún problema de sintaxis en la configuración al relanzarlo.

- Con la nueva configuración, comprobar que si se pausa un contenedor balanceado y esperamos el tiempo configurado para que Traefik lo detecte, en el panel de administración dicho contenedor aparecerá como no disponible y efectivamente traefik no le enviará nuevas peticiones hasta que vuelva a estar operativo
- Si ahora volvemos a activar el contenedor (*docker unpause*), transcurrido el tiempo definido, el balanceador lo detectará y volverá a enrutarle peticiones.

Paso 2.- Protección frente a aumentos inesperados de la demanda o ataques distribuídos de denegación de servicio (DDoS)

Aunque el balanceador sea capaz de repartir la carga entre los frontales balanceados, debe tratar de asignarles una carga adecuada que estos puedan asumir dependiendo de los recursos para los que están configurados. En casos en que esta demanda aumente por encima de la posibilidades del sistema, ya sea de forma legítima o como consecuencia de un ataque, el balanceador debería evitar que llegaran dichas peticiones a los nodos balanceados o al menos que dichas peticiones pudieran atender de forma equitativa a los diferentes usuarios que trataran de conectarse al sistema.

De hecho, un caso particular de esta circunstancia es debida a una variante de los ataques de denegación de servicio, DoS, que persigue anular la disponibilidad del recurso atacado realizando una gran cantidad de peticiones que bloqueen dichos recursos que de otra forma podrían ser utilizados por usuarios legítimos del sistema. Los balanceadores de carga disponen de algunas medidas que pueden mitigar en parte este tipo de ataques, entre ellas podemos citar:

- Limitación del número máximo de conexiones por nodo balanceado

[19]Al tratarse de un servicio web **es importante que la comprobación sea a nivel L7 (http)** frente a comprobaciones L4 (tcp). El motivo es que un servidor que no responda (e.g. pause) no responderá a las peticiones http, aunque seguirá manteniendo activo el puerto. Por lo tanto, una comprobación del primer tipo lo marcaría como no disponible mientras que si es del segundo tipo seguiría considerándolo como activo.

- Limitación del número de peticiones máximas por unidad de tiempo (*rate limit*) desde una misma ip
- Tiempo máximo por conexión (*keep-alive max time*)
- Tiempo de espera (*timeout*) máximo para mantener conexiones entrantes inactivas

Estudiar las opciones de configuración correspondientes que aporta Traefik y adjuntarlas a su configuración.

Nota: en Traefik las limitaciones de peticiones a los nodos balanceados se realizan usando determinados middlewares que deben configurarse como etiquetas (label) y posteriormente asignarse a cada nodo[20] en el fichero compose.yml. Las restricciones de tiempos de espera para conexiones entrantes, por el contrario, se configuran en el apartado de entrypoints del fichero de configuración de balanceador. Si se realizan cambios en este fichero, debe recargarse el balanceador (restart) para que surtan efecto.

Comprobar que las opciones funcionan de forma adecuada realizando tests de carga con la utilidad *'ab'* como se mostró en la actividad 2. Si la configuración es adecuada se observará un incremento del número de peticiones rechazadas (con códigos de estado diferentes a 2xx) además de mostrarse mensajes del balanceador en los logs del contenedor. Observar el impacto de las restricciones introducidas en los recursos utilizados por los contenedores implicados en la salida de la utilidad *docker stats o ctop*

Para la **entrega de la actividad** se proporcionarán los siguientes materiales:

- Breve **informe** (3-4 páginas) donde se expliquen las opciones de configuración del balanceador para la configuración de las opciones solicitadas, mostrando cómo se han incluido en la configuración, y se aporten evidencias: capturas de pantalla, logs, comentarios, etc. que reflejen que se ha conseguido desplegar y hacer funcionar correctamente el sistema.
- Fichero comprimido de la **carpeta del proyecto** con todos los recursos necesarios para poder reproducir su funcionamiento: diferentes ficheros Dockerfile que sean necesarios, compose.yml, fuentes de la aplicación desplegada y ficheros de configuración adicionales necesarios para personalizar y lanzar los diferentes microservicios utilizados. Además, se incluirá un fichero README.md con instrucciones para construir, lanzar y probar el acceso a la aplicación.

En esta actividad **se valorará** tanto el estudio y uso de las opciones de configuración solicitadas como los resultados finales obtenidos de las pruebas realizadas.

Tiempo estimado: 1-2 horas (considerando el tiempo de recopilación de información y realización de las pruebas referidas)
(**plazo de entrega: 29 de octubre**)

[20] Observar ejemplo de configuración del middleware *ipallowlist* en despliegue de prueba en el apartado de preparación

Referencias recomendadas

Los detalles de características y configuración de Traefik, se pueden consultar en principio en la propia web del proyecto. No obstante, existen multitud de referencias en la red donde se tratan en mayor o menor medida los diferentes aspectos de configuración propuestos.

Para la creación de imágenes docker, uso básico de docker y docker compose, se recomienda consultar las diapositivas sobre "Creación y ejecución de Contenedores", así como las referencias incluidas al final de los mismos. No obstante, se recomienda utilizar alguna herramienta para gestionar las operaciones sobre los contenedores, como es el caso de la extensión de docker para vscode o el programa para terminal *lazydocker*, ya que facilitarán bastante el proceso de experimentación.